Table of Contents

1 Introduction....1
1.1 Introduction into UML state diagrams....1
1.1.1 Initial state....1
1.1.2 Final State....2
1.1.3 Entry Point....2
1.1.4 Exit Point....2
1.1.5 Events....2
1.1.6 States....2
1.1.7 Transitions....3
1.1.8 Composite State....3
1.1.9 Concurrent regions....3
1.1.10 Package....4
1.2 Introduction into State transition diagrams....5
2 The Oradio state diagrams views....6
2.1 The OradioMain state diagram....6
2.2 The OradioMain composite state....7
2.2.1 State diagram....7
2.2.2 State table....8
2.3 Other state handler as packages....9
3 Top-down layered static view....10
4 Detailed design of the oradio_control module....11
4.1 Classess....11
4.1.1 Purpose....11
4.1.2 Design & Threading....11
4.1.3 API Reference....11
4.1.3.1 BUTTONS Constants....11
4.1.3.2 Methods:....11
4.1.4 Error handling....11
4.1.5 Attributes for module test....11
4.2 Module test....11

1 Introduction

The oradio_control is based on a state machine.

1.1 Introduction into UML state diagrams

A state diagram models the behavior of a system or an object in response to events. A state diagram mainly consists of states, events, guards, transitions, and actions. It’s a behavioral diagram and it represents the behavior using finite state transitions.

1.1.1 Initial state

It shows the starting point of the system. It’s represented as a filled circle

●

1.1.2 Final State

It shows the ending point of the system. It's represented as a circle with a dot inside

1.1.3 Entry Point

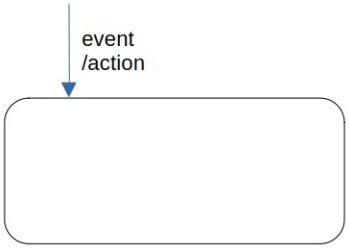
An input connector for control flow into an activity from outside (named) — used when connecting activities across activity partitions or when reusing activity nodes.

1.1.4 Exit Point

An output connector for control flow leaving an activity to outside; used to indicate exit from an activity (named).

1.1.5 Events

An event can be a signal,trigger or a change in the environment of the system. Events may or may not lead to a change in the system's state.

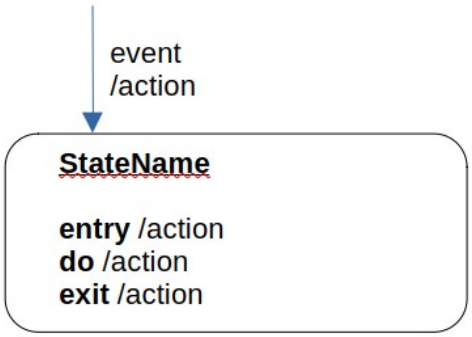


Event /action :

- **event:** trigger that causes the transition.
- **/action**: An operation executed in response to an accepted event

1.1.6 States

A state represents the conditions or circumstances of an object of a class at an instant of time.



- **StateName**: The name of the state
- Use a short, descriptive noun or noun phrase as StateName (e.g., Idle, Processing, Authenticated).
- Make it domain-centric: reflect business meaning
- Be consistent: pick a naming convention (PascalCase, snake_case) and use it everywhere.

State actions are invoked when the system is in a specific state. There are 3 types of state actions:

- **Entry** actions: they are executed when the system enters the state
- **Do** actions: they are continuously executed while the system is in the state
- **Exit** actions: they are executed when the system exits the state

1.1.7 Transitions

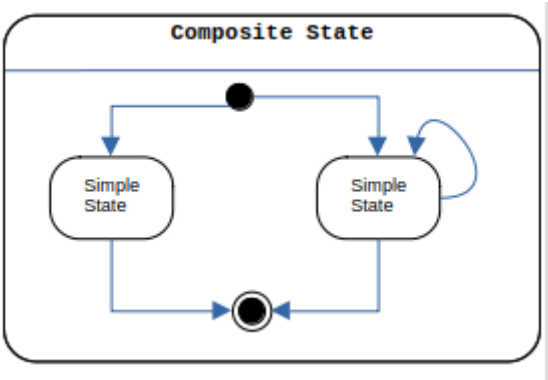
Transitions are shown as arrows from one state to another. They may have three indicators written on them:

- The event that triggers the transition
- The guard statement which is a condition that must be true for the transition to take place
- An action statement that will happen as a result of the transition

```
[isSpotifyConnected]
ButtonPlay
/playSound()
/setLed()
/startSpotifyPlayback()
```

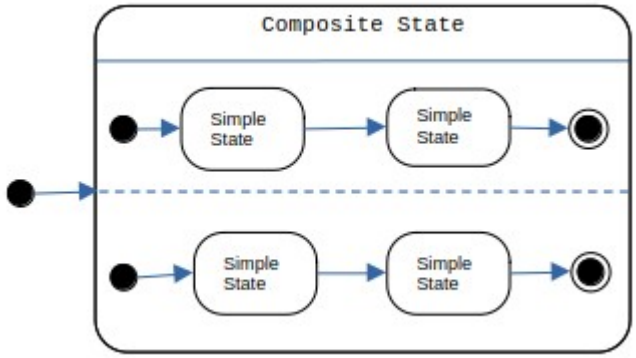
1.1.8 Composite State

A state that has substructure and internal behavior. It can contain one or more regions, which can contain one or more states. The state diagrams are executed in parallel.



1.1.9 Concurrent regions

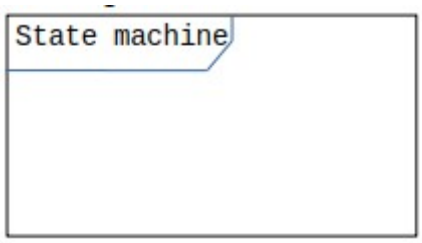
64 Represent separate areas of concern within a state, allowing for independent state transitions in each region, simultaneously.



1.1.10 Package

68 The UML design symbol for a package is depicted as a tabbed folder, which represents a container for grouping related UML elements.

70



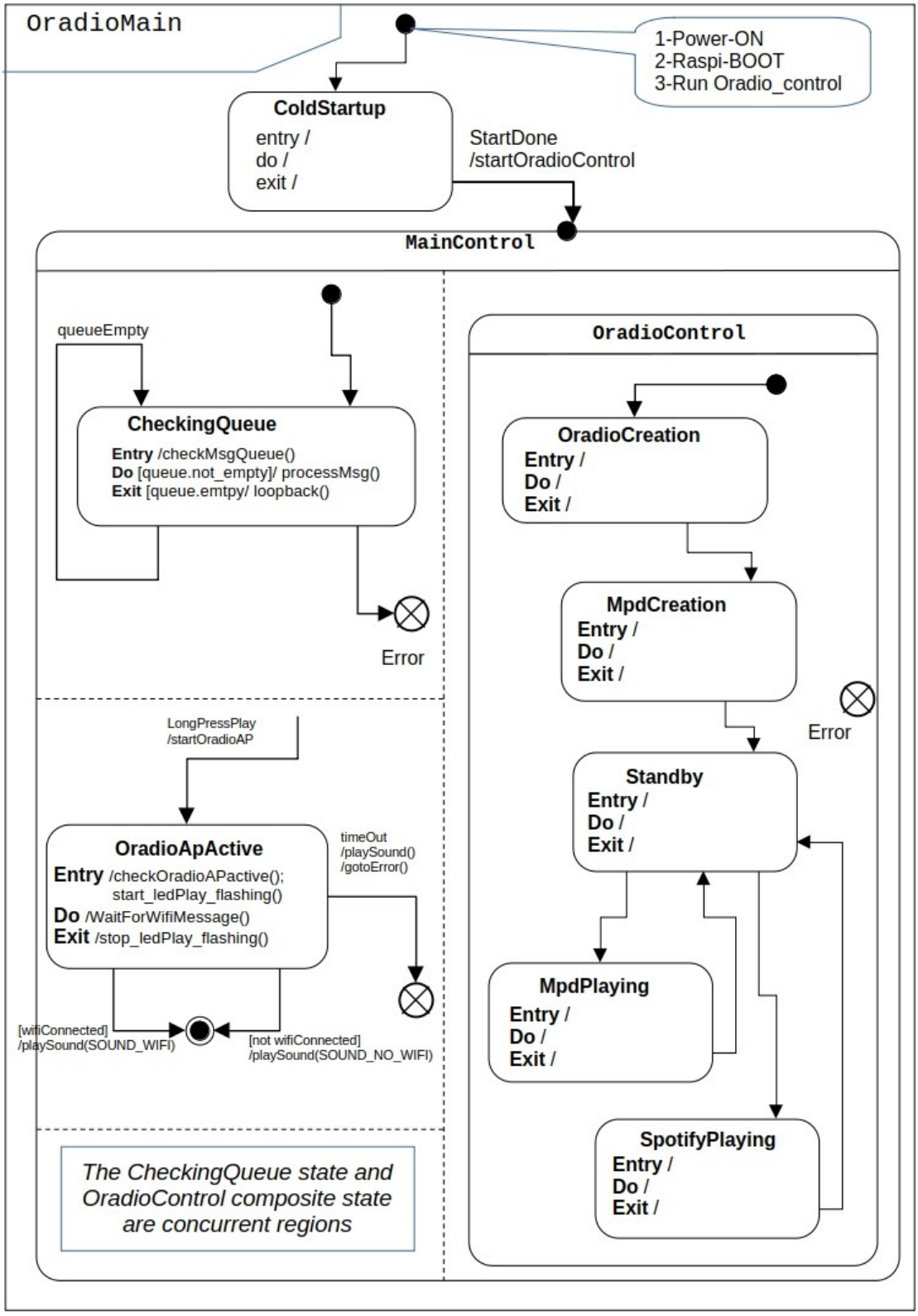
1.2 Introduction into State transition diagrams

a state-transition table is a table showing what state a finit-state machine will move to, based on the current state and other inputs. It is essentially a truth table in which the inputs include the current state along with other inputs, and the outputs include the next state along with other outputs.

Current State	Event	Condition(guards)	Next State	Action

2 The Oradio state diagrams views

2.1 The OradioMain state diagram

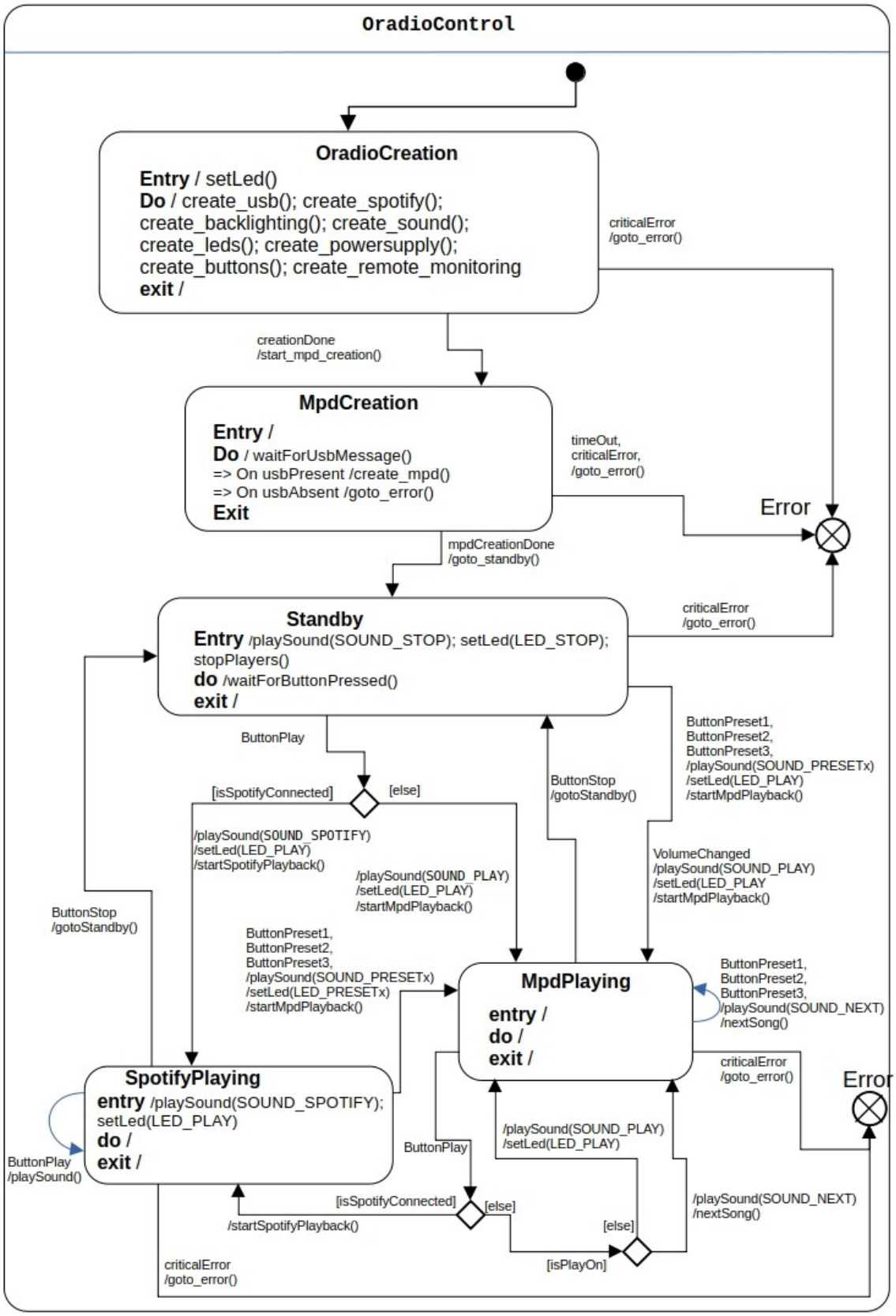


2.1.1 State table

TBD

2.2 The OradioMain composite state

86 ### 2.2.1 State diagram

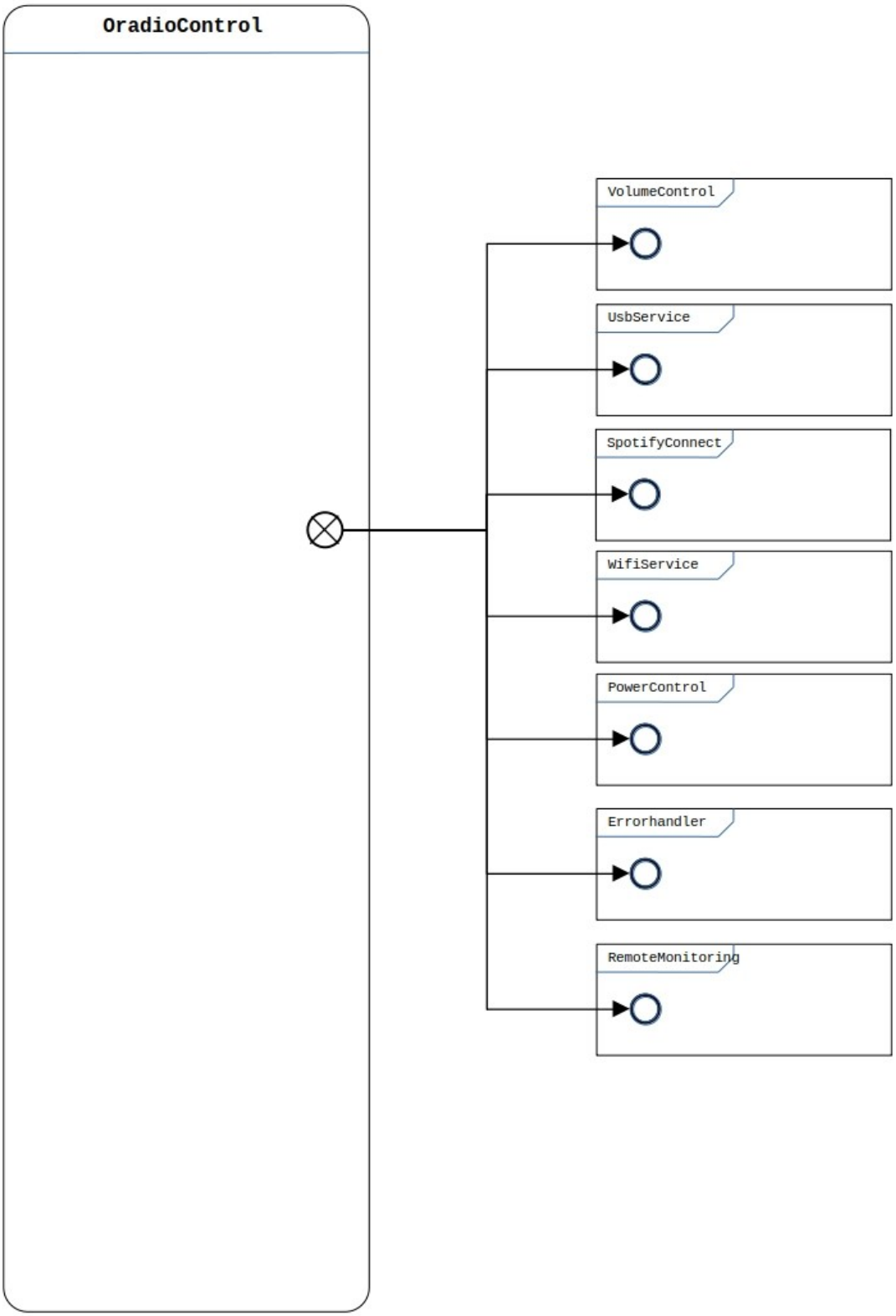


2.2.2 State table

Current State	Event	Condition	Next State	Action
OradioCreation	criticalError	None	ErrorHandling	goto_error()
	creationDone	None	MpdCreation	start_mpd_creation()
MpdCreation	usbAbsent	None	ErrorHandling	goto_error()
	usbPresent	None	Standby	goto_standby()
	timeOut	None	ErrorHandling	goto_error()
	mpdCreationDone	None	Standby	
	criticalError	None	ErrorHandling	goto_error()
Standby	ButtonPreset1, ButtonPreset2, ButtonPreset3	None	MpdPlaying	playSound() setLed() startMpdPlayback()
	VolumeChanged	None	MpdPlaying	startMpdPlayback()
	ButtonPlay	not isSpotifyConnected	MpdPlaying	playSound() setLed) startMpdPlayback()
	VolumeChanged	None	MpdPlaying	startMpdPlayback()
	CriticalError	None	ErrorHandling	goto_error()
	ButtonPlay	isSpotifyConnected	SpotifyPlaying	playSound() setLed() startSpotifyPlayback()
MpdPlaying	ButtonPlay	isSpotifyConnected	SpotifyPlaying	
		not isSpotifyConnected isPlayOn	MpdPlaying	nextSong()
		not isSpotifyConnected not isPlayOn	MpdPlaying	playSound() setLed()
	ButtonStop	None	Standby	gotoStandby()
	CriticalError	None	ErrorHandling	goto_error()
SpotifyPlaying	ButtonStop	None	Standby	gotoStandby()
	CriticalError	None	ErrorHandling	goto_error()

m

2.3 Other state handler as packages



3 Top-down layered static view

TBD

4 Detailed design of the oradio_control module.

4.1 Classess

4.1.1 Purpose

4.1.2 Design & Threading

4.1.3 API Reference

4.1.3.1 BUTTONS Constants

4.1.3.2 Methods:

4.1.4 Error handling

4.1.5 Attributes for module test

4.2 Module test